

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

ASSIGNMENT 1 REPORT | COCSC741 - IRS

Document Indexing and Search Engine

Inverted index and N-gram search for a small document collection.

Harshdip Saha

Roll No. 2023UCA1897

Information Retrieval Systems, COCSC741 - IRS

Submitted to
Department of Computer Science and Engineering
Academic Session 2026-27

PROJECT SCOPE

Document search with two indexes.

This submission extends the retrieval-workbench branch of the upstream project with a clearer preprocessing boundary, updated index statistics and an accessible comparison summary. Users can paste text or upload multiple `.txt` files, then search the same collection with an inverted index and an N-gram index.

> **Live app:** https://document-indexing-search-engine.vercel.app
> **GitHub:** https://github.com/HarshdipSaha/document-indexing-search-engine

Q1: Mini Document Search Engine



Text preprocessing

Words are lowercased, tokenized, stripped of stop words, then stemmed with Porter Stemmer. For example, `running` becomes `run`.

Inverted index

The index stores `term → {document id: frequency}`. This gives direct keyword lookup and term-frequency ranking.

Document updates

New text is saved, indexed and shown in the UI without rebuilding the whole app manually.

Result details

Each result shows title, preview, query-term frequency and total score. So the ranking is visible.

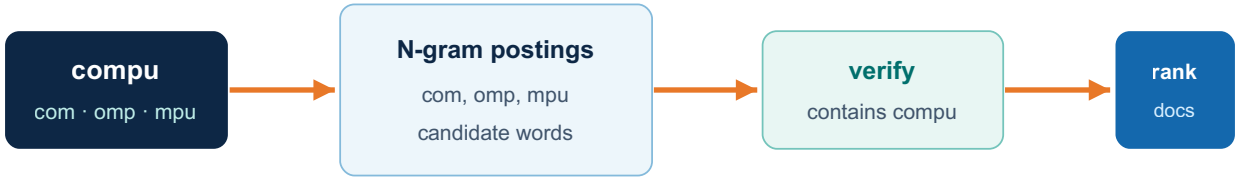
Example index

```
comput  →  {doc_01: 2, doc_04: 1}
retriev →  {doc_02: 3}
index   →  {doc_01: 1, doc_03: 2}
```

Q2: MULTIPLE INDEXING TECHNIQUES

Stemmed keywords and word fragments.

The second index handles cases where the user only remembers part of a word. Typing `compu` should still find `computer`, `computers` and `computing`.



How the N-gram search works

1. Store lowercase raw words with their document frequencies.
2. Break the partial word into character N-grams.
3. Intersect the N-gram lists to get candidate words.
4. Check that the complete fragment exists in the candidate word.
5. Aggregate word frequencies by document and rank the matches.

Comparison

Feature	Inverted index	N-gram index
Search type	Exact processed keyword	Partial word / fragment
Example	`computer` → `comput`	`compu` → computer, computing
Main benefit	Fast direct term lookup	Flexible partial search
Trade-off	Requires a matching stem	Needs candidate intersection and checking

RESULTS AND SUBMISSION CHECK

Measured search behavior.

Requirement	Where it appears	Status
Add multiple text documents	Manual add form and multi-file UTF-8 upload	**Done**
Preprocessing	Tokenization, stop words, Porter stemming	**Done**
Matching docs and frequencies	Result cards and document coverage table	**Done**
Automatic updates	Indexes and UI refresh after each add/upload	**Done**
Exact plus partial search	Inverted and N-gram result modes	**Done**
Time comparison	Comparison ledger with median milliseconds	**Done**

Measured comparison

Sample collection: *Computer Science*, *Information Retrieval* and *N-gram Notes*. Each method ran 50 times after warm-up. Times below are median milliseconds from the same collection.

Query	Inverted index	N-gram index	Observed result
`computer`	2 docs, frequencies 4 and 1 0.0346 ms	1 doc, frequency 3 0.0157 ms	Stemming also finds `computing`.
`compu`	0 docs 0.0277 ms	2 docs, frequencies 4 and 1 0.0182 ms	Partial matching finds computer-related words.

I added coverage for preprocessing, indexing, term frequency, partial matching, uploads, validation, persistence and updates. The project has **23 automated tests**.

```
python -m unittest discover -s tests -p "test_*.py" -v
python -m compileall -q app.py preprocessing.py search_engine.py tests
```

Conclusion

The project keeps both retrieval methods visible. The user can add documents, inspect frequencies, test a fragment and compare the timings in one place.

Submitted by: Harshdip Saha | **Roll No.:** 2023UCA1897